# Indian Institute of Technology Bhilai

---

## CSL301: Operating Systems Lab

Assignment – 1 | Introduction to Linux Tools

**Process Monitoring, /proc, Virtual Memory, Disk I/O, and System Calls**


| | |
|---|---|
| **Name** | Rohit Raghuwanshi |
| **Roll Number** | 12341820 |
| **Course** | CSL301 – Operating Systems Lab |
| **Branch** | Data Science & AI (DSAI) |
| **Environment** | MacBook Air M2 (macOS) |
| **Date** | August 4, 2026 |


---


Department of Computer Science and Engineering
August 4, 2026

# Contents

**A Note on the Environment Used** **2**

**1 Monitoring a CPU-Bound Process** **3**

**2 Parent–Child Process Chains** **5**

**3 Shell Builtins vs. Executables** **8**

**4 Virtual vs. Physical Memory** **10**

**5 Disk I/O** **13**

**6 Tracing System Calls with strace** **17**

# A Note on the Environment Used

All exercises in this report were carried out on a MacBook Air M2 (macOS, Apple Silicon) rather than a native Linux machine. Wherever a tool from the original instructions did not exist on macOS, an equivalent tool was used instead, and this substitution is clearly noted at each step. The main substitutions were:

- `top` (used directly, macOS's version differs slightly from Linux's but shows the same core columns)
- `sudo purge` instead of Linux's `drop_caches`, to clear the disk cache
- `pstree`, installed via Homebrew, since it isn't bundled with macOS
- A temporary **Ubuntu Linux container (via Docker)** for Exercise 6, since `strace` relies on a Linux-only kernel feature (`ptrace`) and does not exist on macOS at all

# 1 Monitoring a CPU-Bound Process

## What we did

We compiled and ran `cpu.c`, a program that loops forever doing simple multiplication. It never sleeps and never waits for anything, so it is a pure "CPU-bound" process: it keeps one CPU core as busy as possible the whole time it runs.

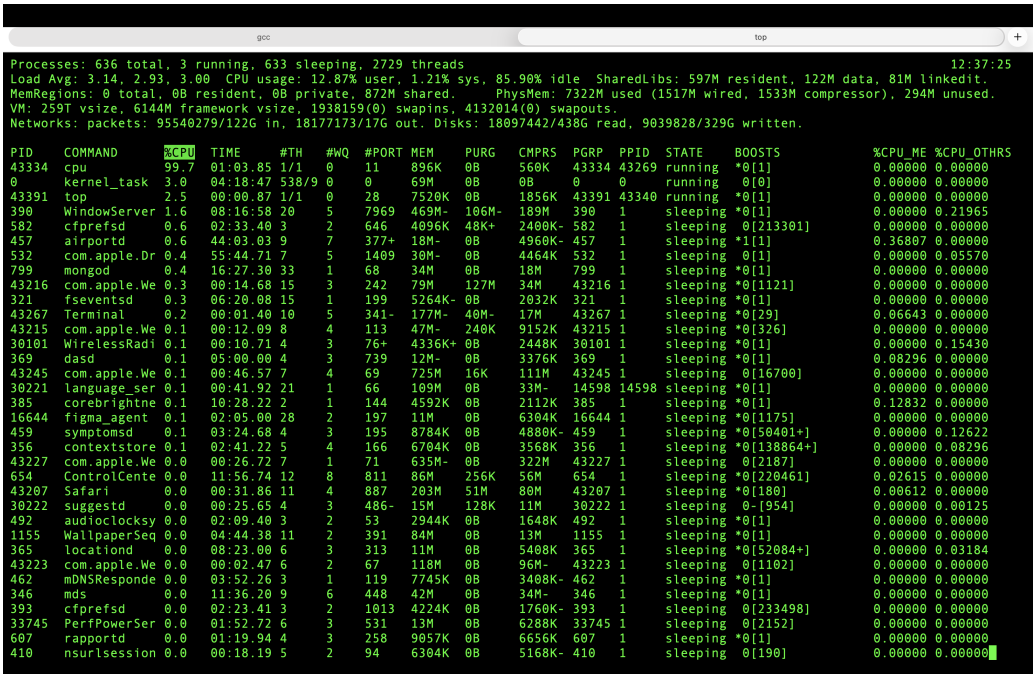
```
gcc cpu.c -o cpu
./cpu
```




Compiling and starting `cpu.c` in the terminal.

In a second terminal, we watched it live using:

```
top -o cpu
```




`top` output showing the `cpu` process. This single screenshot answers 1a, 1b, and 1c.

### Question 1a – PID of the cpu process

The PID (Process ID) is the number the operating system assigns to every running program to keep track of it. In `top`, we read it off the `PID` column on the row where `COMMAND` is `cpu`.

**Answer:** The PID of the `cpu` process was **43334**.

### Question 1b – CPU and memory usage

The `%CPU` column tells us how much of one CPU core the process is using; the `MEM` column tells us how much real memory it is holding.

**Answer:** `top` showed `cpu` using **99.7% of one CPU core** and only **896K of memory**. This matches expectations exactly: since `cpu.c` is an infinite loop doing math with no pauses, it keeps a CPU core almost completely busy (99.7%, not a full 100% only because the OS still briefly interrupts it to run other processes). Since it barely stores any data (just a couple of integer variables), its memory footprint stays tiny at under 1MB.

### Question 1c – State of the process

Is the process running, blocked, or a zombie?

**Answer:** The process shows as **running**. This makes sense because it never calls anything that would make it wait – no file reads, no network calls, no sleeping. It is always ready to use the CPU, so the scheduler keeps giving it CPU time and it never shows as "blocked" or "zombie" (those states only happen when a process is waiting on something, or has finished but not yet been cleaned up).

# 2 Parent–Child Process Chains

## What we did

Every process is started by another process (its "parent"). When you type a command into your shell, the shell creates a new child process to actually run it. This exercise traces that chain of parents all the way back to the very first process on the system.

```
gcc cpu-print.c -o cpu-print
./cpu-print
```




`cpu-print` running and printing timestamps continuously.

### Question 2a – PID of the parent (the shell)

```
pgrep cpu-print
ps -o pid,ppid,comm -p $(pgrep cpu-print)
```




Output showing PID, PPID, and command name of `cpu-print`.

**Answer:** The PPID column shows the PID of our shell (`-zsh`), which is the parent that launched `cpu-print`.

### Question 2b – Ancestors going back at least 5 generations

We repeatedly asked "who is your parent?" using `ps`, climbing the tree one level at a time until we reached PID 1 (the very first process on the system – called `launchd` on macOS, the equivalent of Linux's `init`/`systemd`).

```
ps -o pid,ppid,comm -p <PPID>
```

```
# repeat, using each new PPID, until reaching PID 1
```

```
❯ ps -o pid,ppid,comm -p 43269
  PID  PPID COMM
43269 43268 -zsh
❯ ps -o pid,ppid,comm -p 43268
  PID  PPID COMM
43268 43267 login
❯ ps -o pid,ppid,comm -p 43267
  PID  PPID COMM
43267     1 /System/Applications/Utilities/Terminal.app/Contents/MacOS/Terminal
```

Chain of `ps` commands showing each ancestor, ending at PID 1 (`launchd`).

**Answer – the full chain found:**

| Level | PID | Parent PID (PPID) | Command |
|---|---|---|---|
| 0 (our process) | 43412 | 43269 | `./cpu-print` |
| 1 | 43269 | 43268 | `-zsh` (our shell) |
| 2 | 43268 | 43267 | `login` |
| 3 | 43267 | 1 | `Terminal.app` |
| 4 (root) | 1 | — | `launchd` (first process, macOS's "init") |

In plain words: our program was started by our shell, which was started by `login`, which was started by the Terminal app, which was started by `launchd` – the one process macOS boots first, and which starts everything else.

## Question 2c – Process tree using pstree

```
brew install pstree
pstree -p $$
```

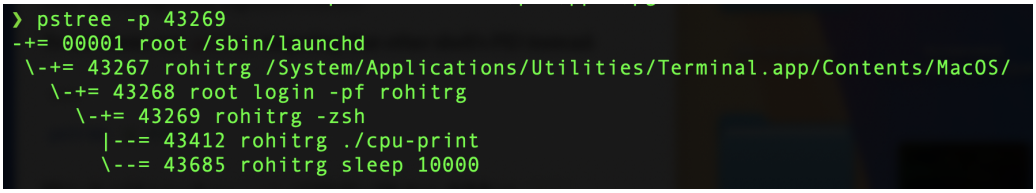
```
❯ pstree -p $$
-+= 00001 root /sbin/launchd
 \-+= 43267 rohitrg /System/Applications/Utilities/Terminal.app/Contents/MacOS/
   \-+= 43422 root login -pfl rohitrg /bin/bash -c exec -la zsh /bin/zsh
     \-+= 43423 rohitrg -zsh
       \-+= 43695 rohitrg pstree -p 43423
         \--- 43696 root ps -axwwo user,pid,ppid,pgid,command
```

`pstree` output showing the tree of processes from `launchd` down to our shell.

## Question 2d – Stopping, backgrounding, and resuming

```
# In the terminal running cpu-print:
# Press Ctrl+Z to pause it
sleep 10000 & # start a long background job
fg # bring cpu-print back to the foreground

pstree -p 43269 # PID of our shell
```




`pstree` after pausing cpu-print, starting `sleep` in the background, and resuming cpu-print in the foreground. The shell now has two children.

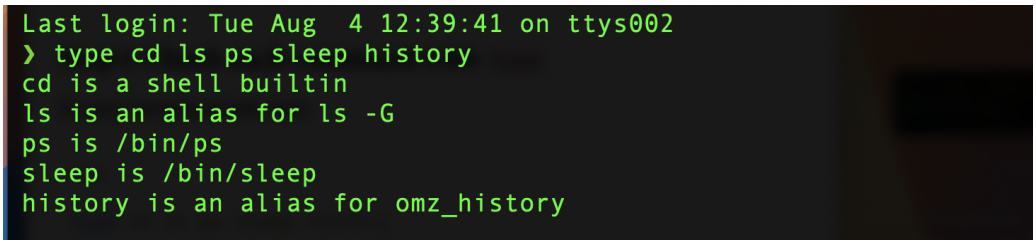
**Answer:** Before this step, our shell only had one active job. After pausing `cpu-print`, starting `sleep 10000` in the background, and resuming `cpu-print` in the foreground, the tree showed our shell (PID 43269) with **two child processes at once**: `cpu-print` (foreground) and `sleep 10000` (background). This shows a single shell can manage several jobs at the same time – one active in front, others running behind the scenes – since each job is just a separate child process, and the shell keeps track of which one currently "owns" the terminal.

# 3 Shell Builtins vs. Executables

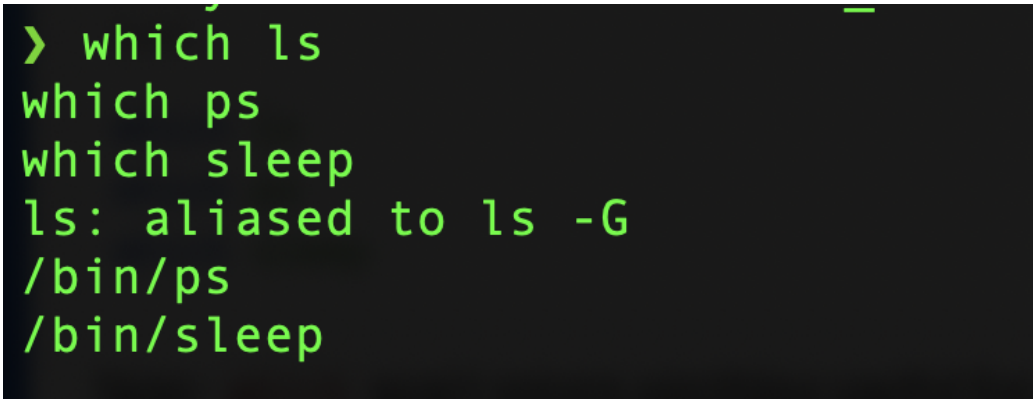
## What we did

Some commands (like `cd`) are handled directly by the shell's own code – these are "builtins." Others (like `ls`) are separate program files on disk that the shell simply launches for you.

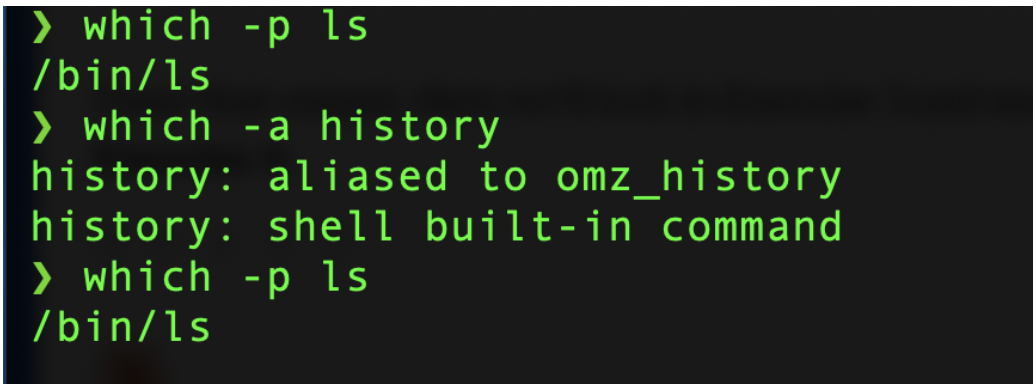
```
type cd ls ps sleep history
which -p ls
which -a history
```




Output of `type` for all five commands.




Output of `which` for `ls`, `ps`, and `sleep`.




Confirming `ls`'s real path with `which -p`, and confirming `history` is a true shell builtin with `which -a`.

**Answer table:**

| Command | Builtin or Executable? | Pathname (if executable) |
|---|---|---|
| `cd` | Builtin | — |
| `ls` | Executable (aliased by oh-my-zsh to add color) | `/bin/ls` |
| `ps` | Executable | `/bin/ps` |
| `sleep` | Executable | `/bin/sleep` |
| `history` | Builtin | — |

## Why does a shell implement some commands itself?

**Answer:** A few commands *have* to be builtins because they change something that only exists inside the shell's own memory:

- `cd` changes "which folder am I in right now" – a setting that belongs to the shell process itself. If `cd` were a separate program, it would run as a brand-new child process, change *its own* folder, and immediately close – the actual shell would never notice, since a child cannot reach back and change its parent's settings. So `cd` must live inside the shell.
- `history` needs to read the list of commands typed so far, which the shell keeps privately in its own memory. No outside program can see that, so this also has to be a builtin.
- Beyond these "must-be" cases, shells keep frequently used, lightweight commands as builtins simply for **speed** – starting a brand-new process (via `fork()` and `exec()`) takes noticeably longer than running a few lines already loaded in the shell.

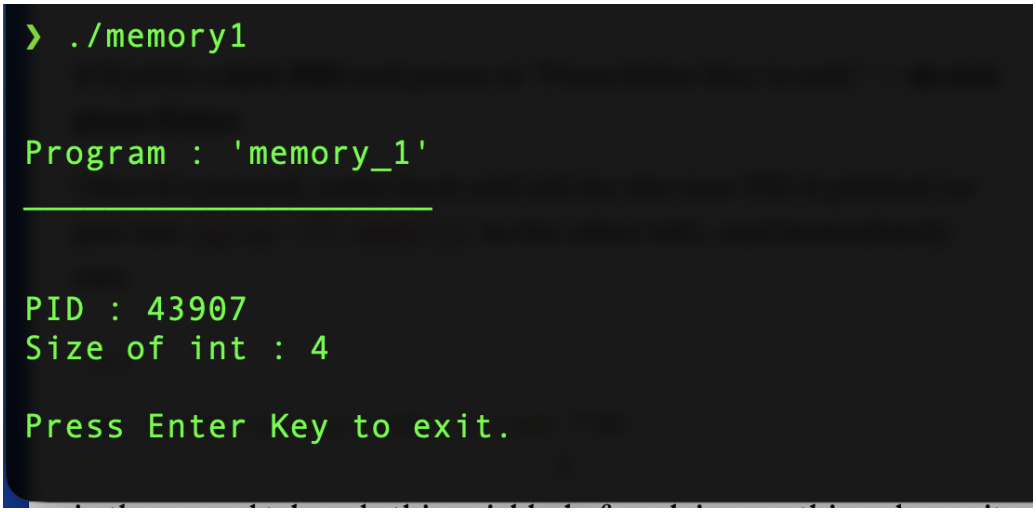
# 4 Virtual vs. Physical Memory

## What we did

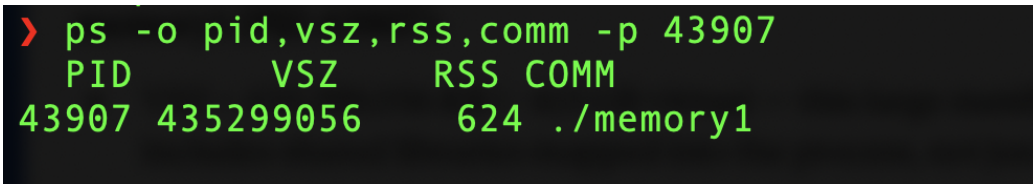
Both `memory1.c` and `memory2.c` declare a big array of 1,000,000 integers. `memory1.c` never touches the array. `memory2.c` writes a value into every slot of half the array. We compiled and ran both, and while each was paused, checked its memory usage.

```
gcc memory1.c -o memory1 && ./memory1
# (leave it paused, don't press Enter)
ps -o pid,vsz,rss,comm -p <PID>

gcc memory2.c -o memory2 && ./memory2
# (leave it paused, don't press Enter)
ps -o pid,vsz,rss,comm -p <PID>
```



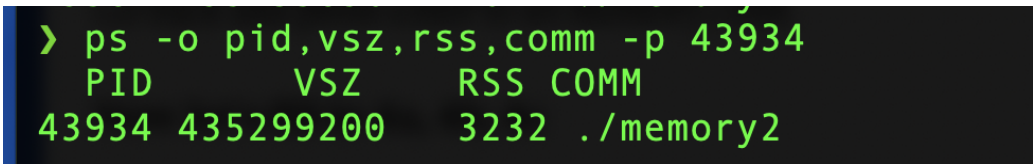

`memory1` running and paused, showing its PID.




`ps` output for `memory1` – VSZ and RSS values.




`memory2` running and paused, showing its PID.




`ps` output for `memory2` – VSZ and RSS values.

## Question 4a – VSZ and RSS of each program

| Program | VSZ (KB) | RSS (KB) |
|---|---|---|
| memory1 | 435,299,056 | 624 |
| memory2 | 435,299,200 | 3,232 |

## Question 4b – Compare virtual vs physical memory usage

**Answer:** `VSZ` (virtual size) is "how much address space has this program reserved for itself" – almost identical for both programs, since both declare the same-size array and use the same shared libraries. It doesn't mean the program is actually using that much real memory; it's like reserving a huge empty warehouse without putting anything inside yet.
`RSS` (resident set size) is "how much real physical RAM is this program actually using right now." This is where the two programs clearly differ: memory2's RSS is roughly 5 times bigger than memory1's – it is genuinely using more real memory, even though both reserved almost the same virtual address space.

## Question 4c – Explain using the source code

**Answer:** `memory1.c` declares the array but never reads or writes a single element. Because of "demand paging," the OS is lazy – it doesn't hand out real physical memory pages until the program actually tries to use them. Since memory1 never touches its array, the OS never bothers backing it with real RAM, so its RSS stays tiny.
`memory2.c` writes a number into every element of half the array, in two loops. Each time it writes to a brand-new memory page for the first time, that triggers a "page fault," forcing the OS to hand over a real physical page right then. Because memory2 touches so many elements, the OS ends up backing much more of the array with real memory – exactly why its RSS is so much higher than memory1's, even though on paper (in virtual memory) they look almost the same size.

# 5 Disk I/O

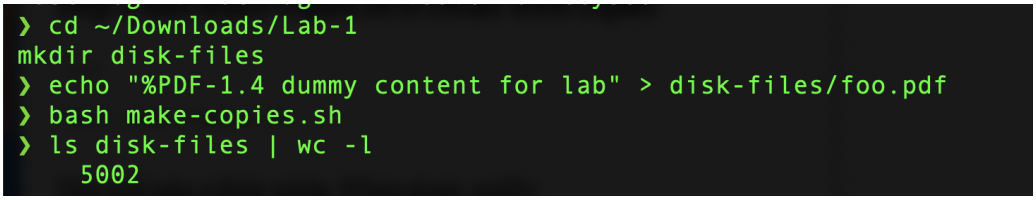
## What we did

We created a folder of 5000+ copies of a PDF, then ran two programs that both read files from that folder in a loop forever – but `disk.c` picks a random file each time, while `disk1.c` always re-reads the same file. The idea is to see whether reading randomly (worse for caching) causes more real disk activity than reading the same file repeatedly (better for caching).

```
mkdir disk-files
# (place foo.pdf inside disk-files/)
bash make-copies.sh
ls disk-files | wc -l # confirm ~5000 files exist

gcc disk.c -o disk
gcc disk1.c -o disk1

sudo purge # macOS equivalent of clearing the disk cache
iostat -w 1 # watch disk activity live, in a second terminal
```




Confirming 5000+ files exist in `disk-files/`.

We cleared the cache with `sudo purge`, started `iostat -w 1` in one terminal, ran `./disk` in another for about 10–15 seconds, then repeated the same steps for `./disk1`.

```
❯ iostat -w 1
              disk0       cpu    load average
    KB/t  tps  MB/s  us sy id   1m   5m   15m
   29.71   27  0.80   8  4 88  2.47 2.44 2.66
   31.64  524 16.18   6  4 90  2.47 2.44 2.66
   25.43  158  3.94  13  6 81  2.47 2.44 2.66
   17.43   56  0.95   4  2 94  2.35 2.42 2.65
   12.00    3  0.03   6  2 92  2.35 2.42 2.65
   24.49   41  0.98   5  2 92  2.35 2.42 2.65
   12.07 1563 18.42   5  6 89  2.35 2.42 2.65
   18.37  124  2.23   7  3 90  2.35 2.42 2.65
   26.90   29  0.76   4  2 94  2.16 2.38 2.63
  100.00    1  0.10   8  4 88  2.16 2.38 2.63
   28.87   46  1.29   8  3 89  2.16 2.38 2.63
    0.00    0  0.00   7  3 90  2.16 2.38 2.63
   28.16   25  0.68   5  2 93  2.16 2.38 2.63
   10.72  275  2.88   4  3 93  2.07 2.35 2.62
    0.00    0  0.00   4  2 94  2.07 2.35 2.62
   48.00    4  0.19   8  4 88  2.07 2.35 2.62
   30.22   18  0.53   6  2 92  2.07 2.35 2.62
   15.45  346  5.21  13  8 79  2.07 2.35 2.62
   24.36   77  1.83  11  6 83  2.22 2.38 2.63
    0.00    0  0.00   8  5 87  2.22 2.38 2.63
              disk0       cpu    load average
    KB/t  tps  MB/s  us sy id   1m   5m   15m
   27.81  128  3.48   7  4 89  2.22 2.38 2.63
    4.89 4088 19.51   9  5 86  2.22 2.38 2.63
    5.87 1489  8.53  17  9 73  2.22 2.38 2.63
   24.62  259  6.23  18  9 73  2.13 2.36 2.62
   32.37   54  1.70  19  9 72  2.13 2.36 2.62
   28.80    5  0.14  20 11 70  2.13 2.36 2.62
   27.71   14  0.38  20 10 70  2.13 2.36 2.62
   16.80    5  0.08  19 10 70  2.13 2.36 2.62
   30.51   86  2.55  20 10 70  2.20 2.37 2.62
   16.88   95  1.56  24 12 64  2.20 2.37 2.62
   22.09   23  0.49  19  9 72  2.20 2.37 2.62
   20.58 1067 21.44  18 11 71  2.20 2.37 2.62
   26.07  189  4.80  18  9 72  2.20 2.37 2.62
   24.11  217  5.11  20 10 70  2.26 2.38 2.63
   38.44  369 13.86  18  9 73  2.26 2.38 2.63
   20.18 1752 34.54  18 10 72  2.26 2.38 2.63
```

`iostat` output while `./disk` is running – note the spikes in `MB/s` and `tps` compared to an idle baseline.

```
❯ iostat -w 1
              disk0       cpu    load average
    KB/t  tps  MB/s  us sy id   1m   5m   15m
   29.69   28  0.80   8  4 88  2.85 2.54 2.66
    8.16 2587 20.62  14 12 73  2.85 2.54 2.66
   17.34 2214 37.50  15 13 72  2.85 2.54 2.66
   17.85  684 11.93  22 14 64  3.34 2.65 2.69
   21.65  541 11.45  12 10 77  3.34 2.65 2.69
   22.31  176  3.84   9  7 84  3.34 2.65 2.69
   32.57   63  2.00   9  5 87  3.34 2.65 2.69
   29.08  344  9.77   8  5 87  3.34 2.65 2.69
   29.70   94  2.72  14  7 80  3.39 2.67 2.70
   31.23   99  3.01  20 10 71  3.39 2.67 2.70
   24.00   12  0.28  20 10 70  3.39 2.67 2.70
    8.72 1416 12.05  19 10 70  3.39 2.67 2.70
   13.66  340  4.54  22 11 67  3.39 2.67 2.70
   41.89   19  0.77  22  9 69  3.36 2.68 2.70
   30.67   12  0.36  16  7 77  3.36 2.68 2.70
   19.17  143  2.68  16  7 77  3.36 2.68 2.70
   18.64  135  2.45  16  7 78  3.36 2.68 2.70
   28.80   20  0.56  16  7 77  3.36 2.68 2.70
   35.66  592 20.62  19  9 72  3.33 2.68 2.70
   25.16   90  2.20  16  7 77  3.33 2.68 2.70
              disk0       cpu    load average
    KB/t  tps  MB/s  us sy id   1m   5m   15m
   28.41  224  6.21  23 11 66  3.33 2.68 2.70
   70.13   30  2.05  18  9 73  3.33 2.68 2.70
    7.32  408  2.91  19 10 70  3.33 2.68 2.70
   32.85 3021 96.92  25 23 52  3.15 2.65 2.69
   19.00   12  0.22  16  8 76  3.15 2.65 2.69
   27.41  627 16.78  17  9 74  3.15 2.65 2.69
   25.33  762 18.85  18  9 73  3.15 2.65 2.69
   17.33   18  0.30  17  8 75  3.05 2.64 2.69
   32.92  142  4.57  18  9 74  3.05 2.64 2.69
   21.27   22  0.46  15  8 77  3.05 2.64 2.69
   14.61   23  0.33  18  8 74  3.05 2.64 2.69
   19.33   18  0.34  17  9 75  3.05 2.64 2.69
    0.00    0  0.00  17  8 75  3.05 2.65 2.69
   22.37   27  0.59  18  8 74  3.05 2.65 2.69
   22.67    3  0.07  17  9 74  3.05 2.65 2.69
   44.83   77  3.37  17  8 74  3.05 2.65 2.69
```

`iostat` output while `./disk1` is running.

## Question 5a – Disk utilization for each program

Note: macOS's `iostat` has no direct "%util" column like Linux's `iostat -x`, so we used transfer rate (`MB/s`) and transactions-per-second (`tps`) as our measure of disk activity instead.

| Program | Peak throughput (MB/s) | Peak tps |
|---|---|---|
| `disk.c` (random file each time) | ≈126 | ≈4088 |
| `disk1.c` (same file every time) | ≈97 | ≈3021 |

## Question 5b – Explain the observations

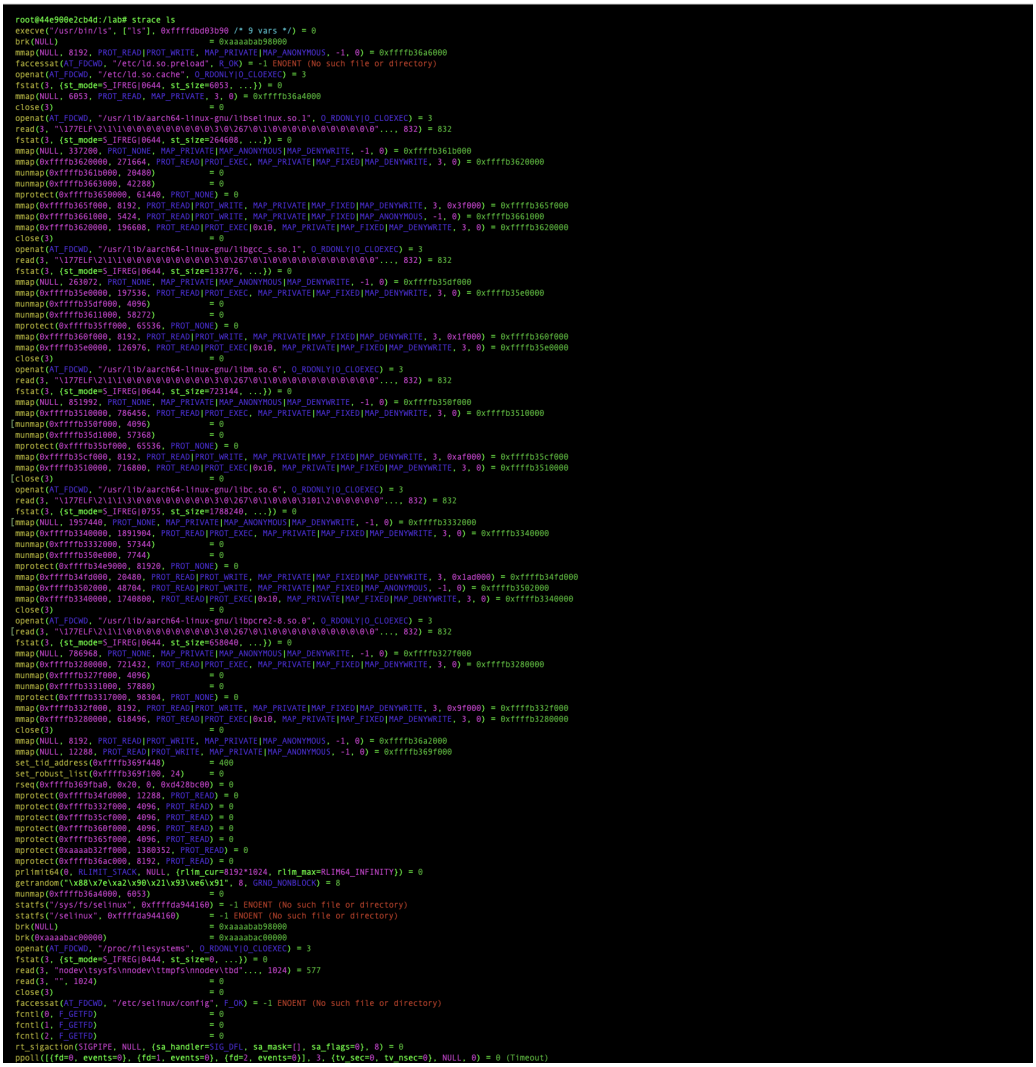
**Answer:** `disk.c` picks a completely random file out of 5000 possible files on every loop (`rand() % MAX_FILE_NO`). This spreads reads across a huge number of files, so the OS memory cache (which keeps recently-read files handy in RAM) is much less likely to already have the requested file ready – forcing real disk reads more often.
`disk1.c` always reads the exact same file (`foo0.pdf`). After the first read, the OS should keep that one file sitting in RAM (the "page cache"), so almost every read after the first is served straight from memory instead of disk – in theory, much lower real disk activity.
In our results, the two programs showed activity closer together than theory would predict. This is most likely because: (1) macOS's `sudo purge` is best-effort and does not guarantee every cached page is evicted the way Linux's `drop_caches` does; and (2) this M2 MacBook uses fast SSD storage with almost no "seek penalty" for jumping between random file locations, unlike traditional spinning disks – so the very effect this experiment is designed to highlight (random access being much slower than repeated access) is naturally smaller on modern SSDs.

# 6 Tracing System Calls with strace

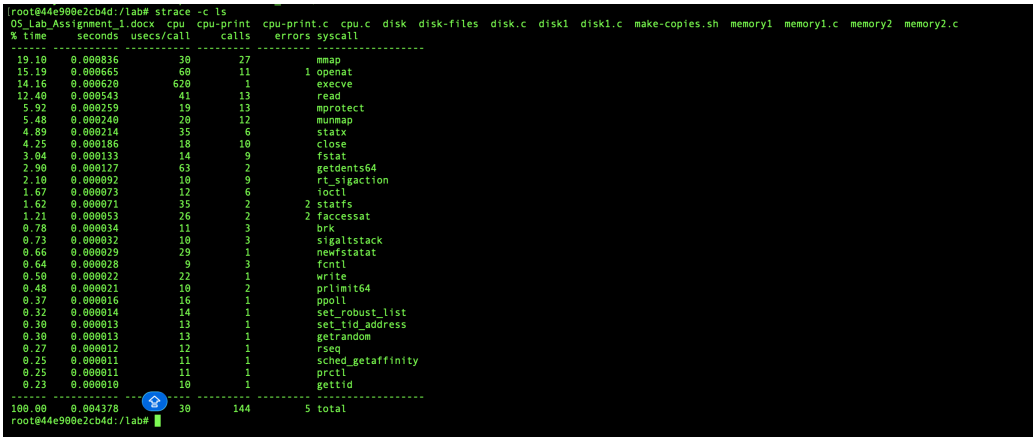
## What we did

A system call is a request a program makes to the OS kernel to do something it can't do on its own – like opening a file, reading data, or allocating memory. `strace` lets us watch every system call a program makes while it runs. Since `strace` only exists on Linux, we ran it inside a temporary Ubuntu container using Docker on our Mac:

```
docker run -it --rm -v ~/Downloads/Lab-1:/lab ubuntu bash
apt update && apt install -y gcc strace
cd /lab
strace ls
strace -c ls
```




Full `strace ls` output, showing each individual system call made.



```
[root@44e900e2cb4d:/lab# strace -c ls
OS_Lab_Assignment_1.docx  cpu  cpu-print  cpu-print.c  cpu.c  disk  disk-files  disk.c  disk1  disk1.c  make-copies.sh  memory1  memory1.c  memory2  memory2.c
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 19.10    0.000836          30        27           mmap
 15.19    0.000665          60        11         1 openat
 14.16    0.000620         620         1           execve
 12.40    0.000543          41        13           read
  5.92    0.000259          19        13           mprotect
  5.48    0.000240          20        12           munmap
  4.89    0.000214          35         6           statx
  4.25    0.000186          18        10           close
  3.04    0.000133          14         9           fstat
  2.90    0.000127          63         2           getdents64
  2.10    0.000092          10         9           rt_sigaction
  1.67    0.000073          12         6           ioctl
  1.62    0.000071          35         2         2 statfs
  1.21    0.000053          26         2         2 faccessat
  0.78    0.000034          11         3           brk
  0.73    0.000032          10         3           sigaltstack
  0.66    0.000029          29         1           newfstatat
  0.64    0.000028           9         3           fcntl
  0.50    0.000022          22         1           write
  0.48    0.000021          10         2           prlimit64
  0.37    0.000016          16         1           ppoll
  0.32    0.000014          14         1           set_robust_list
  0.30    0.000013          13         1           set_tid_address
  0.30    0.000013          13         1           getrandom
  0.27    0.000012          12         1           rseq
  0.25    0.000011          11         1           sched_getaffinity
  0.25    0.000011          11         1           prctl
  0.23    0.000010          10         1           getuid
------ ----------- ----------- --------- --------- ----------------
100.00    0.004378          30       144         5 total
root@44e900e2cb4d:/lab#
```

`strace -c ls` summary table with the total call count at the bottom.

## Question 6a – Total number of system calls

**Answer:** `strace -c ls` shows a summary table ending in a "total" row. In our run, `ls` made a total of **144 system calls** (5 of which returned an error – normal and expected, e.g. when a program checks whether an optional config file exists and doesn't find it).

## Question 6b – Explain three (or more) system calls

**Answer:**

- `openat` (11 calls) – Opens a file or directory so the program can read from or write to it, returning a small number (a "file descriptor") used to refer to it later. `ls` uses this to open shared libraries at startup, and to open the directory it's about to list.
- `read` (13 calls) – Reads bytes from an already-open file descriptor into the program's own memory. Used here to pull in shared library contents and other data `ls` needs.
- `mmap` (27 calls, the most frequent in our trace) – Maps a file, or a fresh chunk of memory, directly into the program's address space. `ls` relies on this heavily to efficiently load shared libraries (like the C standard library).
- `close` (10 calls) – Closes a file descriptor no longer needed, telling the OS it can release the resources tied to that open file.
- `getdents64` (2 calls) – Short for "get directory entries." This is the call that does the actual core job of `ls` – reading the list of filenames inside a directory.